

DBMS LAB — ULTRA CLEAR SQL SYNTAX

Oracle 21c / SQL*Plus • Quick revision sheet for your first lab assessment

TOPICS COVERED

CREATE • ALTER ADD • ALTER MODIFY • ALTER DROP COLUMN • DROP • TRUNCATE • INSERT • UPDATE • DELETE •
SELECT • WHERE • = > < >= <= <> • AND • OR • BETWEEN • IN • LIKE

1. CREATE TABLE

Create a new table.

SYNTAX

```
CREATE TABLE table_name (  
    column1 datatype,  
    column2 datatype,  
    column3 datatype  
);
```

Example:

```
CREATE TABLE student (  
    id NUMBER,  
    name VARCHAR2(50),  
    marks NUMBER,  
    department VARCHAR2(30)  
);
```

IMPORTANT: Put a comma after every column except the last one.

2. ALTER TABLE — ADD

Add a new column to an existing table.

SYNTAX

```
ALTER TABLE table_name  
ADD column_name datatype;
```

Example:

```
ALTER TABLE student  
ADD email VARCHAR2(100);
```

Remember: ALTER + ADD = add a column.

3. ALTER TABLE — MODIFY

Change the datatype/definition of a column.

SYNTAX

```
ALTER TABLE table_name  
MODIFY column_name datatype;
```

Example:

```
ALTER TABLE student  
MODIFY name VARCHAR2(100);
```

Remember: ALTER + MODIFY = change a column.

4. ALTER TABLE — DROP COLUMN

Remove one column from a table.

SYNTAX

```
ALTER TABLE table_name  
DROP COLUMN column_name;
```

Example:

```
ALTER TABLE student  
DROP COLUMN email;
```

Remember: DROP COLUMN removes only that column.

5. DROP TABLE

Completely remove the table and its data.

SYNTAX

```
DROP TABLE table_name;
```

Example:

```
DROP TABLE student;
```

DROP = table structure + data are removed.

6. TRUNCATE TABLE

Remove ALL rows but keep the table structure.

SYNTAX

```
TRUNCATE TABLE table_name;
```

Example:

```
TRUNCATE TABLE student;
```

TRUNCATE = all data removed, table remains.

7. INSERT

Add a new row.

SYNTAX

```
INSERT INTO table_name  
VALUES (value1, value2, value3);
```

Example:

```
INSERT INTO student  
VALUES (101, 'Arjun', 85, 'CSE');
```

Text uses single quotes. Numbers do not need quotes.

8. INSERT — SELECTED COLUMNS

Insert values into only the columns you specify.

SYNTAX

```
INSERT INTO table_name (column1, column2)
VALUES (value1, value2);
```

Example:

```
INSERT INTO student (id, name, marks)
VALUES (105, 'Karan', 79);
```

The columns and values must match in order.

9. UPDATE

Change existing data.

SYNTAX

```
UPDATE table_name  
SET column_name = value  
WHERE condition;
```

Example:

```
UPDATE student  
SET marks = 95  
WHERE id = 101;
```

VERY IMPORTANT: WHERE decides which rows are changed.

10. UPDATE — MULTIPLE COLUMNS

Change more than one column in one query.

SYNTAX

```
UPDATE student  
SET marks = 95,  
    department = 'IT'  
WHERE id = 101;
```

Key idea:

```
SET = what to change    |    WHERE = which row(s) to change
```

Without WHERE, every row can be updated.

11. DELETE

Remove selected rows.

SYNTAX

```
DELETE FROM table_name  
WHERE condition;
```

Example:

```
DELETE FROM student  
WHERE id = 101;
```

DELETE + WHERE = remove matching rows only.

12. DELETE — ALL ROWS

Remove every row but keep the table.

SYNTAX

```
DELETE FROM student;
```

Check:

```
SELECT * FROM student;
```

Without WHERE, DELETE removes all rows.

13. SELECT — ALL COLUMNS

Display every column.

SYNTAX

```
SELECT * FROM student;
```

Meaning:

```
SELECT * = show all columns
```

Use this to quickly see all data in a table.

14. SELECT — SPECIFIC COLUMNS

Display only the columns you need.

SYNTAX

```
SELECT name, marks  
FROM student;
```

Meaning:

```
Only NAME and MARKS are displayed.
```

General form: SELECT column1, column2 FROM table_name;

15. WHERE

Filter rows using a condition.

SYNTAX

```
SELECT *  
FROM student  
WHERE marks > 80;
```

Meaning:

Show only students whose marks are greater than 80.

WHERE = choose which rows to display.

16. COMPARISON OPERATORS

Use these with WHERE.

SYNTAX

=	equal to
>	greater than
<	less than
>=	greater than or equal
<=	less than or equal
<>	not equal

Example:

```
SELECT *  
FROM student  
WHERE marks >= 80;
```

Learn all six operators.

17. AND

Both conditions must be true.

SYNTAX

```
SELECT *  
FROM student  
WHERE department = 'CSE'  
AND marks > 80;
```

Meaning:

CSE AND marks above 80.

AND makes the filter more specific.

18. OR

At least one condition must be true.

SYNTAX

```
SELECT *  
FROM student  
WHERE department = 'CSE'  
OR department = 'ECE';
```

Meaning:

```
CSE OR ECE .
```

OR gives more possible matches.

19. BETWEEN

Find values inside a range. Both limits are included.

SYNTAX

```
SELECT *  
FROM student  
WHERE marks BETWEEN 70 AND 90;
```

Meaning:

Marks 70, 71, ... 90 are included.

BETWEEN 70 AND 90 includes 70 and 90.

20. IN

Match one of several values.

SYNTAX

```
SELECT *  
FROM student  
WHERE department IN ('CSE', 'ECE', 'IT');
```

Meaning:

```
Department is CSE OR ECE OR IT.
```

IN is a short way to write multiple OR conditions.

21. LIKE

Search text using patterns.

SYNTAX

```
SELECT * FROM student WHERE name LIKE 'R%';
```

```
SELECT * FROM student WHERE name LIKE '%a';
```

```
SELECT * FROM student WHERE name LIKE '%r%';
```

```
SELECT * FROM student WHERE name LIKE '_r%';
```

Patterns:

```
'R%' = starts with R    |    '%a' = ends with a    |    '%r%' = contains r    |    '_  
r%' = second character is r
```

% = zero or more characters. _ = exactly one character.

22. MUST REMEMBER FOR THE LAB

Command	What it does
CREATE TABLE	Creates a table
ALTER ... ADD	Adds a column
ALTER ... MODIFY	Changes a column
ALTER ... DROP COLUMN	Removes a column
INSERT	Adds rows
UPDATE ... WHERE	Changes selected rows
DELETE ... WHERE	Deletes selected rows
DELETE FROM table	Deletes all rows; table remains
TRUNCATE TABLE	Deletes all rows; table remains
DROP TABLE	Deletes the entire table
SELECT	Displays data
WHERE	Filters rows

SQL*PLUS HELPFUL COMMANDS

```
DESC student;
```

```
SELECT * FROM student;
```

```
EXIT;
```

LAB RULES: Use commas in CREATE TABLE. Use single quotes for text. Use WHERE when UPDATE/DELETE should affect selected rows. End SQL statements with ;